

What happens when Software Grows?
Teams change/ Requirements change/ deadlines shift/ Codebases that were small grow massive/ Adding features becomes dangerous/ Fixing bugs introduces new ones/ Developers are afraid to touch old codebases.

Symptoms of Bad Design
Rigidity - Difficult to change/ *Fragility* - Easy to break/ *Immobility* - Hard to reuse/ *Viscosity* - Easier to hack than to do properly/ *Opacity* - Hard to understand what the code does.

History
Introduced by Robert C. Martin in 2000. Named and popularized by Michael Feathers. Built on decades of OO design. Focused on writing code which are flexible, extensible & maintainable.
SOLID principles are the foundation for Agile development. Critical for Test-Driven Development (TDD)/ Design Patterns/ Clean Architecture.

Acronym
S - Single Responsibility Principle **O** - Open/Closed Principle **L** - Liskov Substitution Principle **I** - Interface Segregation Principle **D** - Dependency Inversion Principle
Single Responsibility Principle
Definition: "A class should have one, and only one, reason to change." With multiple responsibilities are more fragile and harder to maintain.

In simple words:
A class should be responsible for just one role. Not mix unrelated responsibilities together.
Bad Example: Employee Class
class Employee {
 void calculatePay() { ... } //Problem: Business logic (salary calculation)/
 void saveToDatabase() { ... } //Persistence logic (database save)/Presentation
 void generateReport() { ... } //Logic (report generation) All mixed.
}

Problems When SRP is Violated
Tight Coupling: Change in database affects business logic/ *Higher Risk:* Bug fixes introduce unexpected side-effects/ *Hard Testing:* Unit testing is complicated because responsibilities are tangled/ *Hard Reuse:* You can't reuse parts without dragging unrelated code. [Violating lead to fragile, tangled systems]

How to Apply SRP
Split responsibilities into separate classes: One class per role/ Clear boundaries of responsibility/ Changes in one area don't affect others/ Design Tip: Whenever you see "and" in a class description, it probably needs a split!
Good Example: Separate Classes

```
class Employee { ... }  
class PayCalculator {  
    double calculatePay(Employee e) { ... } //Each class now has one reason to  
                                           //change  
}  
class EmployeeRepository {  
    void save(Employee e) { ... }  
}  
class EmployeeReport {  
    void generate(Employee e) { ... }  
}
```

Checklist: Signs SRP Might Be Violated
Class does many unrelated things/ Class grows too large over time ("God Class")/ Class touches too many external systems (DB, UI, Email, etc.)/ Multiple teams need to modify the same class for different reasons. **When in doubt: Split it out!**

Open/Closed Principle
"Software entities should be open for extension, but closed for modification." Meaning: Add new behavior without changing existing code/ Protect working code from being broken by changes.

In simple words: You should be able to extend the behavior of a class without altering its source code. Existing tested code should stay untouched as much as possible. Think: Build systems like LEGO blocks – add more pieces without reshaping old ones!
Core modules stay stable. New features plug in without breaking old ones.

Bad Example: Graphic Editor with Switch Case
class GraphicEditor {
 void drawShape(Shape s) { //Adding new shapes requires editing
 if (s.type == "Circle") drawCircle(s); //this method – violating OCP!
 else if (s.type == "Square") drawSquare(s);
 }
}

Problems When OCP is Violated
Fragile – changing switch logic may break unrelated shapes./ *Risky* – one typo could crash the system./ *Inefficient* – recompile and retest old working code every time./ *Bottleneck* – one team must control all modifications. *Bad for scalability and team growth.*

How to Apply OCP → Polymorphism
Create an interface or abstract class/ Let each new behavior implement or inherit. Core classes call the abstract methods – not care about details. This way, new behaviors come as new classes – no modification needed.

Good Example: Using Polymorphism
interface Shape { void draw();
 class Circle implements Shape {
 public void draw() { /* draw circle */ }
 }
 class Square implements Shape { //New shapes extend the system - Without
 public void draw() { /* draw square */ } //touching GraphicEditor!
 }
 class GraphicEditor {
 void drawShape(Shape s) {
 s.draw();
 }
 }

Checklist: Signs OCP Might Be Violated
Frequent modifications to stable code when adding new features/ Growing long "if-else" or "switch-case" chains/ Risk of introducing bugs in old logic when adding new options/ New types require editing core processing classes. **When in doubt: Abstract it out!**

Liskov Substitution Principle
Definition: "Subtypes must be substitutable for their base types without altering desirable behavior."
In simple words:

If code works with a base class, it should also work with any derived class – without surprises. Subclasses must honor the promises made by their parents. Think: Child objects should behave like parents – or better.
Bad Example: Rectangle and Square
class Rectangle {
 void setWidth(int w) { ... } //Square "breaks" the behavior expected behavior of
 void setHeight(int h) { ... } //Rectangle class
}

class Square extends Rectangle {
 void setWidth(int w) { super.setWidth(w); super.setHeight(w); }
 void setHeight(int h) { super.setHeight(h); super.setWidth(h); }
}

Problems When LSP is Violated
Clients get unpredictable behavior/ Testing becomes complicated/ Bugs are introduced silently – harder to detect. **Violation destroys confidence in type hierarchy!**

How to Apply LSP
Redesign the hierarchy carefully/ Do not use inheritance if behavior cannot be preserved/ Prefer interfaces or separate abstractions if substitution doesn't make sense/ Tip: Inheritance = "is-a" relationship – check it carefully.
Good Example: No Forced Inheritance
interface Shape { int area();
 class Rectangle implements Shape {
 int width, height;
 int area() { return width * height; } //Both shapes implement the same
 } //contract without confusion
 class Square implements Shape {
 int side;
 int area() { return side * side; }
 }

Checklist: Signs LSP Might Be Violated
Subclass overrides methods and changes expected behavior/ Subclass throws unexpected exceptions/ Clients must add "instanceof" checks to distinguish subclasses.

When in doubt: Flatten the hierarchy!
A Duck class extends Bird but cannot fly (throws exception when fly() is called).

Interface Segregation Principle
Definition: "Clients should not be forced to depend upon interfaces that they do not use." Prefer many small, focused interfaces over one large general-purpose interface.
In simple words: Clients should only know about the methods that are relevant to them. Don't burden classes with unnecessary obligations. Think: Don't make a printer implement "fax" if it can't fax! In Code: Clients should only see the methods they need
Bad Example: Multi-Function Interface
interface Machine {
 void print(); //Classes are forced to implement
 void scan(); //methods they don't support!
 void fax();
}

class BasicPrinter implements Machine {
 public void print() { ... }
 public void scan() { throw new UnsupportedOperationException(); }
 public void fax() { throw new UnsupportedOperationException(); }
}

Problems When ISP is Violated
Unnecessary code complexity/ Higher risk of runtime errors (e.g., unsupported methods)/ Harder to understand and maintain/ Break interfaces into smaller, focused roles!

How to Apply ISP
Split interfaces by responsibilities: One interface per logical group/ Classes implement only what they actually support. Example: Printer, Scanner, and Fax interfaces separately.

Good Example: Focused Interfaces
interface Printer { void print(); }
interface Scanner { void scan(); } //Classes now only implement what they
interface Fax { void fax(); } //truly offer!
class BasicPrinter implements Printer {
 public void print() { ... }
}

Checklist: Signs ISP Might Be Violated
Classes throw exceptions for unimplemented methods/ Interfaces seem bloated or unrelated/ Clients know too much about unrelated behavior.

When in doubt: Split it out!
Dependency Inversion Principle (DIP)
Definition:
High-level modules should not depend on low-level modules. Both should depend on abstractions. Also, abstractions should not depend on details. Instead, details should depend on abstractions.

In simple words: Don't make your main logic depend directly on concrete classes. Instead, depend on interfaces or abstract classes so you can swap implementations easily.

Think: Plug the appliance into the wall socket, not directly wired to the power plant!
In Code: High-level code should talk to interfaces, not concrete implementations.
Bad Example (Violation of DIP)
class EmailService {
 public void sendEmail(String message) {
 // send email...
 }
}

class Notification {
 private EmailService emailService = new EmailService(); // Direct dependency!

 public void alert(String msg) {
 emailService.sendEmail(msg);
 }
}

Hard to replace Email with SMS/Push notifications later.
Notification is tightly coupled to EmailService.
Unit testing becomes harder.

How to Apply DIP
Create an interface that defines the behavior.
High-level class depends on the interface.
Concrete classes implement the interface.

Good Example (Follows DIP)
// 1. Abstraction (Interface)
// This defines the general behavior: sending a message.
// It does NOT specify *how* the message is sent.
interface MessageService {
 void sendMessage(String message);
}

// 2. Concrete Implementation 1
// This class sends messages via Email.
// It follows the MessageService contract.
class EmailService implements MessageService {
 public void sendMessage(String message) {
 System.out.println("Sending Email: " + message);
 }
}

// 3. Concrete Implementation 2
// This class sends messages via SMS.
class SMSService implements MessageService {
 public void sendMessage(String message) {
 System.out.println("Sending SMS: " + message);
 }
}

// 4. High-Level Class (Depends on Abstraction, NOT Concrete Classes)
class Notification {
 private MessageService messageService; // Reference to interface, not implementation

// Dependency is injected via the constructor (Dependency Injection)
public Notification(MessageService messageService) {
 this.messageService = messageService;
}

// High-level behavior: alert user
// It calls the interface method,
// not knowing the concrete sending method.
public void alert(String msg) {
 messageService.sendMessage(msg);
}

public class Main {
 public static void main(String[] args) {
 // Choose Email Service
 MessageService email = new EmailService();

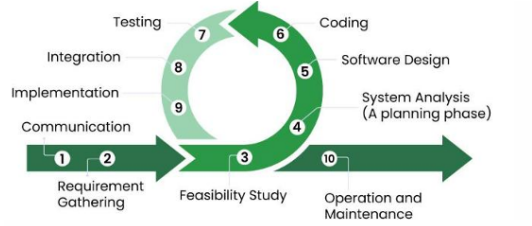
// Inject EmailService into Notification
Notification notification = new Notification(email);

// Send notification
notification.alert("Your order has been shipped!");
}

Advantages:
Swap implementations easily (Email ↔ SMS ↔ Push)
More flexible, testable, and maintainable
High-level logic does not change when low-level implementations change
Checklist: Signs DIP Might Be Violated
Your class uses new a lot to create other objects.
High-level logic changes because of low-level details.
Hard to replace a component with a mock in tests.
One change cascades into many code changes.

When in doubt: Depend on interfaces, not concrete classes! Keep your code flexible and future-proof.

Software Development Process
Modularity, which involves dividing a software system into smaller, more manageable components or modules. By breaking the system into modules, developers can work on different components at the same time, boosting productivity and minimizing the risk of errors.
Fortran and COBOL, among the first programming languages, revolutionized scientific computing and business applications. These languages offered a *higher level of abstraction*, making programming more accessible to a wider range of users. C programming language and its derivatives (C++ C#). Due to their *efficiency and low level control* became the preferred languages for *system-level programming*. Java revolutionized *cross-platform development* with its "write once, run anywhere" philosophy.
Software crisis highlighted the need for a more disciplined approach to managing software development. Structured programming emerged as the solution, focusing on using clear and modular code structures to reduce complexity and enhance maintainability.
Software Development Process consists of a series of steps focusing on designing, building, testing, and maintaining software applications. This usually involves stages such as planning, design, coding, testing, deployment, and continuous maintenance to ensure the software meets user requirements and operates as intended.



Importance of SDP: [Software is typically developed for four main reasons,
1. To Solve a Problem: Software is created to address specific challenges or inefficiencies in various industries, making processes more efficient or providing solutions to common problems.
2. To Meet User Needs: Software is often developed with the goal of fulfilling specific needs or demands from users, whether for personal use or for businesses to improve their operations.
3. To Improve Productivity: Many software applications are designed to help individuals or organizations work more efficiently, automate repetitive tasks, or streamline workflows.
4. To Innovate or Create New Opportunities: Some software is developed to push the boundaries of technology, introducing new features, services, or creating entirely new markets or possibilities.]
Software Development Life Cycle a structured approach to software development that outlines the stages and processes involved in creating, deploying, and maintaining software. Ensures the production of high-quality software that meets user requirements, is delivered on time, and is cost-effective. Is a foundational concept in software development that ensures a methodical approach to creating software systems. While the specific processes or models can vary, the core principle remains the same.



1. Requirement Gathering and Analysis
Purpose: Understand and document the needs and requirements of the stakeholders (customers, users, and business units). **Activities:** Conduct interviews, surveys, or focus groups with stakeholders, Define functional and non-functional requirements, Create a Software Requirements Specification (SRS), Epics, User Stories, **Outcome:** A detailed and approved requirements document that serves as a foundation for the entire development process.

2. System Design
Purpose: Translate the requirements into a detailed blueprint for the software, including architecture, databases, and user interfaces. **Activities:** High-Level Design (HLD): Defines the system architecture, including technology stack, modules, and interfaces, Low-Level Design (LLD): Provides detailed designs for each module, including database schema, UI/UX design, algorithms, etc., Use design tools like UML, flowcharts, ER diagrams (ERDs), and wireframes. **Outcome:** Design documents that provide clear instructions for developers on how to implement the system.

3. Implementation (Coding)
Purpose: Develop the software according to the design documents. **Activities:** Write code for the system based on the approved design, Follow coding standards and best practices to ensure readability, maintainability, and scalability, Perform unit testing on individual modules. **Outcome:** Source code and compiled software that fulfill the system design.

4. Testing
Purpose: Ensure the software is free of bugs and meets the requirements, expectations set by stakeholders and end users. **Activities:** Conduct various levels of testing (unit testing, integration testing, system testing, acceptance testing), Identify and fix defects, Validate that the system meets functional and non-functional requirements. **Outcome:** A bug-free system that is ready for deployment. Test reports are generated as evidence.

5. Deployment
Purpose: Release the software to the production environment for end-users/live-users. **Activities:** Deploy the software on the target environment (could be a server, cloud, or end-user devices), Perform user acceptance testing (UAT) in the production environment to ensure everything works as expected, Provide training and documentation to end-users as needed. **Outcome:** The software is successfully deployed and accessible to users.

6. Maintenance
Purpose: Ensure the software continues to function correctly after deployment, and address any issues or updates. **Activities:** Monitor system performance and user feedback, Fix bugs that arise after deployment, Release updates or patches to enhance functionality or address security vulnerabilities, Implement new features based on user requests or business changes. **Outcome:** The system continues to run smoothly, with updates and patches applied over time.

Software Development Life Cycle (SDLC) Model is a structured, systematic framework that defines the complete process of planning, creating, testing, and deploying a software product. It establishes the sequential or iterative phases—such as requirements analysis, design, implementation, testing, and maintenance—along with the corresponding deliverables and review points, serving as a project management blueprint to ensure efficiency, quality assurance, and alignment with business objectives from conception to retirement.

Agile methodology is an alternative to the waterfall model, focused on flexibility, collaboration, and iterative development.

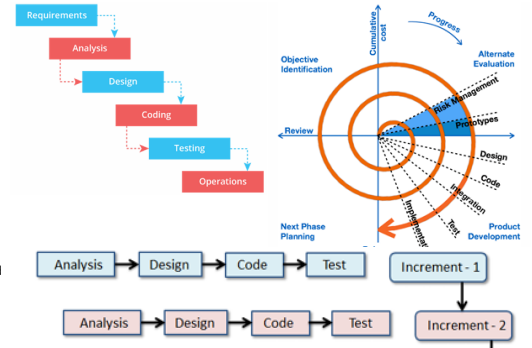
Software Development Models
1. Waterfall Model (1970s)

Characteristics: Linear and sequential, each phase of the software development life cycle (SDLC) is completed before moving to the next. **Phases:** Requirements gathering, system design, implementation, testing, deployment, and maintenance. **Limitations:** Inflexible to changes, especially in the early stages. If issues arise late in the process, it can be costly to address. **When to use:** Clear and Well-Defined Requirements, Small to Medium-Scale Projects, Minimal Customer Involvement, Predictable Outcome.

2. Prototyping Model (1970s-1980s)
Characteristics: Emphasizes the creation of prototypes (early versions) of the software to understand the requirements more clearly. **Advantages:** Better understanding of user requirements through feedback, quick development of a working model. **Limitations:** Prototypes can become too focused on being "perfect," leading to scope creep and potentially wasting resources. When to use: Unclear or Evolving Requirements, Frequent Customer Interaction, Exploratory or Innovative Projects, User Interface Design

3. Incremental Model (1980s)
Characteristics: Modular Development, Continuous Improvement, Flexibility, Partial Deployment **Advantages:** Early Partial Deliverables, Flexibility, Customer Involvement, Risk Management **Limitations:** Requires Good Initial Planning, Integration Challenges, Overlapping Increments, Increased Overhead **When to use:** Project Can Be Broken Into Modules or Subsystems, When Time-to-Market Is Important, Evolving or Growing Requirements, Early Feedback, Ongoing Product Maintenance

4. Spiral Model (1980s)
Characteristics: A hybrid of the Waterfall model and prototyping. It emphasizes risk management and iterative development. **Phases:** Planning, risk analysis, engineering, testing, and evaluation. These phases are repeated in cycles (spirals). **Advantages:** Allows for iterative refinement, with a focus on identifying and mitigating risks early. **Limitations:** Can be complex to manage and requires expertise in risk analysis. **When to use:** High-Risk Projects, Frequent Evaluation, Risk Management, Large and Complex Systems, Prototype Evolution, Continuous Client Feedback, Long-Term, Ongoing Development



Production to Agile

Agile is a set of principles for software development under which requirements and solutions evolve through collaborative effort. Focus on iterative development, customer feedback, and flexibility. Agile is the ability to create and respond to change. It is a method for navigating and ultimately thriving in an unpredictable and challenging environment.

Agile methodology is a project management and software development approach that emphasizes flexibility, collaboration, and customer-centricity.

Four core values of Agile

Individuals and Interactions, over processes and tools [Emphasizes communication and the skills of the team over strict adherence to predefined structures.]

Working software, over comprehensive documentation [“Focuses on delivering functional increments of the product frequently, prioritizing real-world results over paperwork.”]

Customer collaboration, over contract negotiation [Promotes continuous communication with the client throughout the project to ensure the product meets evolving needs.]

Responding to change, over following a plan [Recognizes that requirements will inevitably change and encourages flexibility over rigid adherence to the initial plan.]

Fuller or heavyweight approaches are characterized by detailed upfront planning, extensive documentation, and strict control over the process. These models prioritize predictability and stability over rapid response to change.

Lightweight approaches are characterized by minimal documentation, rapid iteration, and high adaptability. These models prioritize fast delivery of working software and continuous customer feedback.

Agile Principles

The Agile Principles are a set of values and beliefs that guide business leaders in implementing agile methodologies in the workplace.

Customer & Value Priority

(1) Our highest priority is to satisfy the customer through the early and continuous delivery of valuable software.

(2) Welcome changing requirements, even late in development. Agile processes harness change for the customer's competitive advantage.

(3) Deliver working software frequently, from a couple of weeks to a couple of months, with a preference to the shorter time scale.

(4) Business people and developers must work together daily throughout the project.

(5) Build projects around motivated individuals. Give them the environment and support they need, and trust them to get the job done.

(6) The most efficient and effective method of conveying information to and within a development team is face-to-face conversation.

(7) Working software is the primary measure of progress.

(8) Agile processes promote sustainable development. The sponsors, developers, and users should be able to maintain a constant pace indefinitely.

(9) Continuous attention to technical excellence and good design enhances agility.

(10) Simplicity—the art of maximizing the amount of work not done—is essential.

(11) The best architectures, requirements, and designs emerge from self-organizing teams.

(12) At regular intervals, the team reflects on how to become more effective, then tunes and adjusts its behavior accordingly.

Agile Frameworks

1. Scrum [Roles (Product Owner, Scrum Master, Development Team)/ Artifacts (Product Backlog, Sprint Backlog, Increment)/ Ceremonies (Sprint Planning, Daily Standup, Sprint Review, Retrospective).]

2. Kanban [Visual workflow management] Focus on continuous delivery and limiting work in progress (WIP)/ [To Do/ Doing/ Done]

3. Feature-Driven Development (FDD) [Model-driven and feature-oriented approach, Focuses on delivering small, client-valued features, Works well for large and complex projects Uses domain models, feature lists, and short iterative cycles/Roles (Project Manager/Chief Architect/Development Manager/Chief Programmer)/ Artifacts (Domain Object Model/Feature List/Design Packages/Code and Versioned Builds)]

Benefits

Faster delivery of high-quality software/ Improved customer satisfaction/ Adaptability to changing requirements/ Enhanced team collaboration and morale/ Continuous improvement through retrospectives.

Agile Issue Type is a category or label used to classify work in Agile project management tools

1. **Epic Description:** An epic is a large body of work that can be broken down into smaller user stories. It represents a high-level goal or feature. **Example:** “Implement user authentication system.” **Purpose:** Helps organize and track progress on large features or initiatives. [Business perspective]

2. **Story (or User Story) Description:** A user story represents a small, actionable piece of work that delivers value to the end user. It is written from the user's perspective. **Format:** As a [user], I want [action] so that [benefit]. **Example:** “As a customer, I want to reset my password so that I can regain access to my account.” **Purpose:** Helps the team focus on delivering user-centric features.

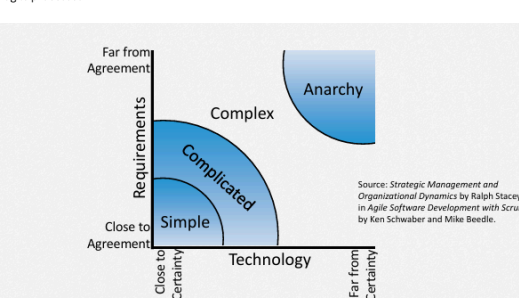
3. **Task Description:** A task is a specific piece of work that needs to be completed as part of a user story or larger project. Tasks are often technical and actionable. **Example:** “Write unit tests for the login module.” **Purpose:** Breaks down user stories into smaller, manageable pieces of work. [Dev Team Pers]

3. **Bug Description:** A bug represents a defect or issue in the software that needs to be fixed. **Example:** “Login button does not respond when clicked.” **Purpose:** Tracks and resolves issues that affect the functionality or user experience. [Test User/ Dev Perspective]

An Introduction to Scrum

Scrum is an agile process that allows us to focus on delivering the highest business value in the shortest time. It allows us to rapidly and repeatedly inspect actual working software (every two weeks to one month). The business sets the priorities. Teams self-organize to determine the best way to deliver the highest priority features. Every two weeks to a month anyone can see real working software and decide to release it as is or continue to enhance it for another sprint.

Characteristics = Self-organizing teams/ Product progresses in a series of month-long “sprints”/ Requirements are captured as items in a list of “product backlog”/ No specific engineering practices prescribed/ Uses generative rules to create an agile environment for delivering projects/ One of the “agile processes”



Sprints (No changes during a sprint) Scrum projects make progress in a series of “sprints”/ Analogous to Extreme Programming iterations/ Typical duration is 2–4 weeks or a calendar month at most/ A constant duration leads to a better rhythm/ Product is designed, coded, and tested during the sprint.

Scrum framework (Roles/ Ceremonies/ Artifacts)

Product owner

Define the features of the product/ Decide on release date and content/ Be responsible for the profitability of the product (ROI)/ Prioritize features according to market value/ Adjust features and priority every iteration, as needed/ Accept or reject work results

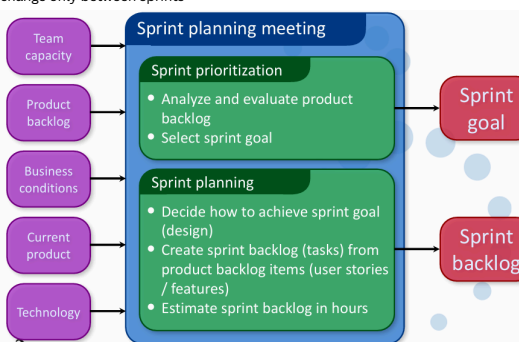
ScrumMaster

Represents management to the project/ Responsible for enacting Scrum values and practices/ Removes impediments/ Ensure that the team is fully functional and productive/ Enable close cooperation among all roles and functions/ Shield the team from external interferences

Team

Typically 5–9 people

Cross-functional/ Programmers, testers, user experience designers, etc./ Members should be full-time/ May be exceptions (e.g., database administrator)/ Teams are self-organizing/ Ideally, no titles but rarely a possibility/ Membership should change only between sprints



Sprint planning Team selects items from the product backlog they can commit to completing/ Sprint backlog is created/ Tasks are identified and each is estimated (1–16 hours)/ Collaboratively, not done by the ScrumMaster/ High-level design is considered

The daily scrum Parameters/ Daily/ 15–minutes/ Stand-up/ Not for problem solving/ Whole world is invited/ Only team members, ScrumMaster, product owner, can talk/ Helps avoid other unnecessary meetings/ Everyone answers 3 questions [What did you do yesterday?/ What will you do today?/ Is anything in your way?/ These are not status for the ScrumMaster/ They are commitments in front of peers]

The sprint review Team presents what it accomplished during the sprint/ Typically takes the form of a demo of new features or underlying architecture/ Informal/ 2-hour prep time rule/ No slides/ Whole team participates/ Invite the world

Sprint retrospective

Periodically take a look at what is and is not working/ Typically 15–30 minutes/ Done after every sprint/ Whole team participates [ScrumMaster/ Product owner/ Team/ Possibly customers and others]/ Whole team gathers and discusses what they'd like to Start/ Stop/ Continue doing This is just one of many ways to do a sprint retrospective.

Product backlog

The requirements → A list of all desired work on the project/ Ideally expressed such that each item has value to the users or customers of the product owner/ Prioritized by the product/ Reprioritized at the start of each sprint

Managing the sprint backlog BURNDOWN Chart is also an artifact

Individuals sign up for work of their own choosing/ Work is never assigned/ Estimated work remaining updated daily/ Any team member can add, delete or change the sprint backlog/ Work for the sprint emerges/ If work is unclear, define a sprint backlog item with a larger amount of time and break it down later/ Update work remaining as more becomes known

Scalability

Typical individual team is 7 ± 2 people/ Scalability comes from teams of teams/ Factors in scaling [Type of application/ Team size/ Team dispersion/ Project duration/ Scrum has been used on multiple 500+ person projects]

Introduction to Project Management

Project management is a temporary, unique endeavor undertaken to create a specific product, service, or result, involving a series of tasks and activities executed within defined constraints like time, budget, and resources.

Project management is the application of knowledge, skills, tools, and techniques to project activities to meet project requirements. It's the practice of planning, organizing, and executing the tasks needed to turn a brilliant idea into a tangible product, service, or deliverable.

Key aspects of project management include, [Defining project scope/ Identifying deliverables/ Managing risks/ Effective communication across teams]

Project management in Software Development Ensures efficient use of resources/ Helps in delivering projects on time and within budget/ Manages risk and scope effectively.

Principles of Project Management 1. Establish the project structure/ 2. Define project goals and objectives/ 3. Identify a project sponsor/ 4. Form roles and responsibilities/ 5. Ensure team accountability/ 6. Manage project scope and changes/ 7. Create a risk management plan/ 8. Monitor progress/ 9. Focus on effective value delivery/ 10. Establish a performance management baseline/ 11. Finalize the project/ 12. Examine successes

Software Process Quality Management

Software Process Quality refers to how well the software development process is defined, managed, measured, and improved to consistently produce high-quality software products. It's not just about the final product, but about how the software is built ensuring that the methods, tools, and practices used in development lead to reliable, maintainable, and efficient software.

Key Aspects of Software Process Quality

1. Well-Defined Processes: Clear steps from planning to deployment. [Using Agile or DevOps practices.]/ 2. Consistency: Repeating successful practices across projects./ 3. Measurement & Monitoring: Tracking progress, defects, code quality, test coverage./ 4. Continuous Improvement: Regular reviews, retrospectives, and upgrades to tools/processes

Why is it important? Ensures High-Quality Software/ Reduces Cost and Rework/ Improves Project Timelines/ Enhances Team Collaboration/ Increases Customer Satisfaction

Objectives of Process Quality Management Deliver reliable and maintainable software/ Meet customer expectations/ Improve team efficiency and reduce rework

Key Concepts Software Process/ Quality Assurance (QA)/ Quality Control (QC)

Quality Management Activities Process Definition/ Process Assessment/ Process Improvement/ Process Monitoring and Control

Software Quality Models

McCall's Quality Model



Product Operation (How well the software works)- Correctness – Does it meet the requirements?- Reliability – Can it work without crashing?- Efficiency – Does it use system resources well?- Integrity – Is it secure from unauthorized access?- Usability – Is it easy to use?

Product Revision (How easy it is to change)- Maintainability – Can you fix bugs easily?- Flexibility – Can you adapt it to new requirements?- Testability – Is it easy to test?

Product Transition (How easy it is to move and reuse)- Portability – Can it work on different platforms?- Reusability – Can parts of it be reused in other software?- Interoperability – Can it work with other systems?

Boehm's Quality Model

To evaluate software quality from a user's perspective, by breaking it down into measurable characteristics.

High-Level Characteristics (User Needs), These represent overall quality goals- As-is utility – Is it useful as delivered?- Maintainability – Can we easily make changes?- Portability – Can we run it elsewhere?

Intermediate-Level Characteristics, These help assess the high-level ones- Reliability – Will it work under expected conditions?- Efficiency – Does it use resources wisely?- Human Engineering – Is it user-friendly?- Testability – Is it easy to test?- Understandability – Can developers understand it?- Flexibility – Can it adapt to changes?

Primitive Characteristics, These are measurable properties like- Response time- Throughput- Documentation clarity- Code structure



Importance of Quality Standards Define benchmarks for acceptable quality. Examples ISO (International Organization for Standardization)/ IEC (International Electrotechnical Commission) 25010 – Software Quality Models

(functionality, usability, reliability, etc.) IEEE Standards – For testing, documentation, and design./ ISO 9001 – Quality management systems. Benefits,

• Improved product quality and customer satisfaction. Enhanced process repeatability and consistency. Better regulatory compliance. Foundation for continuous improvement. In software, it means behaving responsibly, with integrity, and with consideration for the public good.

Builds trust with users, employers, and the public./ Prevents legal consequences (e.g., privacy law violations)./ Ensures team harmony and productive work culture.

Software Process Quality Standards

Quality maturity refers to how well an organization defines, implements, measures, and continuously improves its processes to ensure consistent and high-quality outcomes in software development and service delivery. Assessing maturity helps organizations identify their current capabilities and chart a path for improvement.

SEI CMM (Capability Maturity Model) was developed by the Software Engineering Institute (SEI) to evaluate the maturity of software development processes.

CMMI (Capability Maturity Model Integration) evolved from CMM and provides a more integrated framework applicable beyond software (e.g., systems engineering, product development, services).

Purpose of CMMI [Improve process effectiveness and efficiency/ Ensure repeatable and measurable outcomes/ Align business goals with process improvement/ Promote continuous process improvement]

Key Process Areas (KPAs) are the essential focus areas that an organization must address to improve process capability and move up the maturity levels.

Goals: What needs to be achieved. [Commitment to perform/ Policies and leadership support/ Ability to perform/ Resources and training needed/ Activities performed/ Tasks to be executed/ Measurement and analysis/ How effectiveness is tracked./ Verifying implementation/ How compliance is ensured.]

Maturity Level 1 – Initial (No KPAs)

At this level, processes are unpredictable and reactive./ There are no formal KPAs./ Success is highly dependent on individual efforts and heroics.

Maturity Level 2 – Managed [At this level, the focus is on basic project management practices to track cost, schedule and functionality.]

KPAs – Requirements Management [Ensures that requirements are documented, agreed upon, and controlled throughout the project lifecycle.] Project Planning [Establishes estimates, schedules, and resource plans to guide project execution.] Project Monitoring and Control [Tracks the project's progress and helps take corrective actions when deviations occur.] Supplier Agreement [Manages relationships with external suppliers to ensure quality deliverables.] Measurement and Analysis [Collects and analyzes data to support informed decision-making.] Process and Product Quality Assurance [Ensures that processes are followed and work products meet quality standards.] Configuration Management controls and tracks changes to software and documentation, ensuring integrity, consistency, and traceability across the project.

Maturity Level 3 – Defined [Focus shifts to organization-wide standards and process integration.] Requirements Development [Focuses on eliciting, analyzing, and validating the needs of customers and stakeholders to ensure clarity and completeness.] Technical Solution [Involves designing and developing appropriate solutions that fulfill these requirements.] Product Integration [Ensures that individual components are combined and integrated correctly to form a complete and functioning system.] Verification [Checks that work products meet defined specifications.] Validation [Confirms that the final product fulfills its intended use in the real operating environment.] Organizational Process Focus [builds and maintains the infrastructure for continuous process improvement across the organization.] Organizational Process Definition [provides standardized and documented processes to be used consistently across projects.] Organizational Training [ensures that team members possess the necessary knowledge and skills to perform their roles effectively.] Integrated Project Management [coordinates activities and resources across multiple projects and disciplines to maintain alignment and efficiency.] Risk Management proactively identifies, analyzes, and mitigates potential risks, and Decision Analysis and Resolution uses structured and criteria-based methods to make informed and effective decisions.

Maturity Level 4 – Quantitatively Managed [At this level, the focus is on quantitative measurement and control of processes and product quality. (Organizational Process Performance Collects process data across the organization to establish baselines.) Quantitative Project Management Uses statistical and quantitative techniques to manage project performance and quality.]

Maturity Level 5 – Optimizing This level emphasizes continuous process improvement and defect prevention. (Organizational Innovation and Deployment Introduces and deploys innovations that improve organizational performance.) Causal Analysis and Resolution Identifies root causes of defects and prevents recurrence through process improvements.]

Steps in Assessing Quality Maturity

1. Select Assessment Approach: Choose between SCAMPI (Standard CMMI Appraisal Method for Process Improvement) or internal audits./ 2. Define Scope: Identify which projects, departments, or processes will be assessed./ 3. Collect Evidence: Review documentation, interview staff, and observe processes./ 4. Rate Process Areas: Evaluate each KPA against expected goals and practices./ 5. Determine Maturity Level: Identify the maturity level based on the extent to which KPAs are fulfilled./ 6. Report Findings and Recommendations: Provide a maturity profile and suggestions for improvement.

Benefits of CMMI for Quality Maturity Assessment

Provides a structured roadmap for process improvement/ Helps organizations benchmark themselves against industry standards/ Encourages disciplined engineering practices/ Leads to improved product quality, predictability, and customer satisfaction/ Supports compliance with regulatory and contractual requirements

Introduction to Documentation Refers to written text or illustration that accompanies computer software./ Types: Requirements documentation, design docs, test cases, user manuals, system documentation.

Importance of Documentation Supports communication among stakeholders./ Ensures traceability and accountability./ Aids in maintenance and scalability./ Critical for onboarding and audits.

V&V Techniques [Static Techniques: Reviews, inspections, walkthroughs./ Dynamic Techniques: Testing (unit, integration, system, acceptance)/ Formal Methods: Mathematical proofs and modeling]

Coupling refers to the degree of interdependence between software modules. High coupling means that modules are closely connected and changes in one module may affect other modules. Low coupling means that modules are independent, and changes in one module have little impact on other modules.

Cohesion refers to the degree to which elements within a module work together to fulfill a single, well-defined purpose. High cohesion means that elements are closely related and focused on a single purpose, while low cohesion means that elements are loosely related and serve multiple purposes

Types of Cohesion Coincidental/ Logical/ Temporal/ Procedural/ Communicational/ Sequential/ Functional (Best) **Types of Coupling** Content (worst)/ Common/ Control/ Stamp/ Data/ Message (Best)

Change Management in software engineering is the systematic approach to dealing with changes in software requirements, code, documentation, or environments during the software development lifecycle (SDLC). It ensures that changes are introduced in a controlled, coordinated, and traceable manner. Prevents uncontrolled modifications that can introduce bugs./ Maintains system stability and version integrity./ Helps teams track history, analyze impact, and communicate efficiently./ Ensures regulatory compliance in critical systems (e.g., healthcare, aviation).

Key Components of Change Management Change Request (CR): A formal proposal to make a change (e.g., add a feature, fix a bug)./ Impact Analysis: Assesses how the change will affect other components, time, cost, and risk./ Change Approval- Review by a Change Control Board (CCB) or decision makers./ Change Implementation- Approved changes are planned, coded, tested, and integrated./ Change Verification- Validation through testing and reviews to ensure correctness./ Documentation & Communication- Update related documents, versionlogs, and notify stakeholders.

